

FINAL INTERNSHIP PROJECT ASSIGNMENT

OpsPilot

Autonomous Incident Investigation Agent

Advanced Agentic AI Capstone • 3-Week Delivery

Assignment Objective

Design, build, evaluate, and defend a production-oriented agentic AI system capable of autonomously investigating software incidents using tools, planning, retrieval, reflection, verification, state, and controlled human approval.

Target Level: Internship Final Assessment / Junior Applied Agentic AI Readiness

1. Project Overview

Modern engineering teams investigate production incidents by manually checking metrics, logs, deployment history, runbooks, architecture documents, and previous incidents. OpsPilot must automate this investigative workflow while remaining evidence-grounded, observable, bounded, and safe.

Problem Statement

Given an incident request such as “Investigate why checkout API latency increased in the last two hours,” the system must create an investigation plan, select and execute appropriate tools, maintain state, form and validate hypotheses, critique its own investigation, re-plan when evidence is insufficient, and produce a cited incident report.

Expected Outcome

- Identify a likely root cause and provide an explicit confidence assessment.
- Ground conclusions in metrics, logs, deployments, runbooks, or historical incidents.
- Explain contradictory or missing evidence.
- Recommend next actions.
- Require human approval before high-impact operational actions.

Illustrative Final Response

LIKELY ROOT CAUSE — Example

Deployment checkout-v2.4 introduced repeated database retry calls.

Confidence: 87%

Evidence:

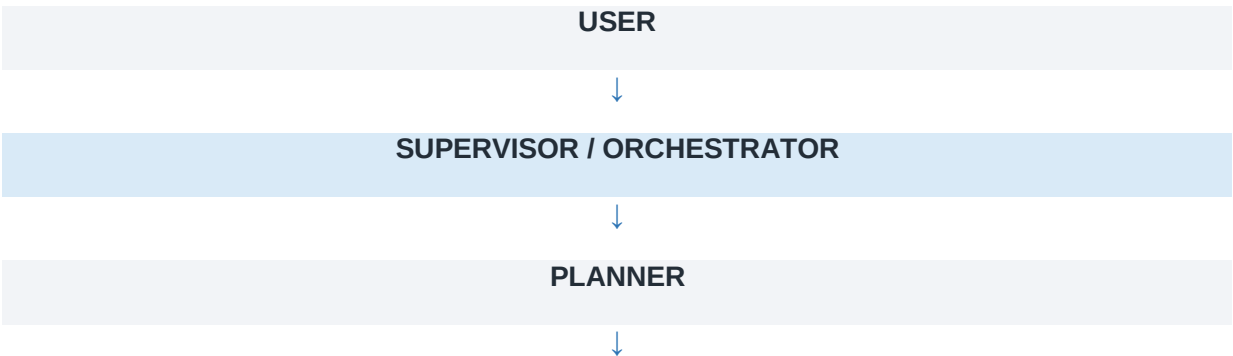
- Latency increased 8 minutes after deployment.
- Database timeout logs increased 420%.
- Historical incident INC-104 showed similar behavior.

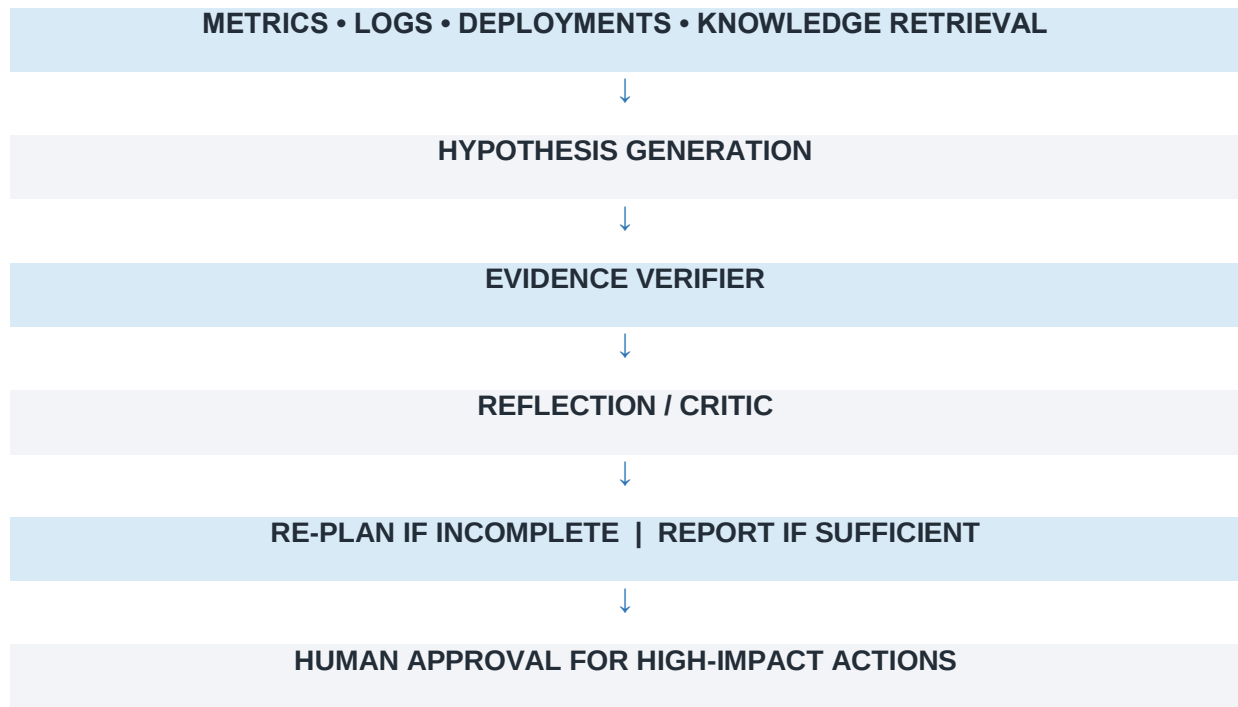
Recommended action: Roll back checkout-v2.4.

Human approval required before rollback.

2. Required Agent Architecture

The exact implementation may vary, but the submitted architecture must clearly represent planning, specialized tool access, hypothesis formation, verification, reflection, re-planning, and approval boundaries.





3. Mandatory Agentic AI Requirements

3.1 Tool Calling

- Implement a minimum of seven tools: `query_metrics`, `search_logs`, `get_deployments`, `search_incidents`, `retrieve_runbook`, `create_incident_report`, and `request_rollback`.
- Tools must use structured schemas and accept meaningful arguments.
- The model must select tools based on user intent and current investigation state.

3.2 Dynamic Tool Registry

- Use a registry or equivalent dispatch mechanism to map model-requested tool names to executable functions.
- Large hardcoded `if/elif` execution chains are not acceptable.

3.3 Multi-Step Agent Loop

- The agent must repeatedly plan or decide, select a tool, execute it, observe the result, and determine the next action.
- The investigation sequence must not be fully hardcoded.

3.4 Planning and Re-Planning

- Generate a structured investigation plan before or during execution.
- The plan must include a goal and ordered or dependency-aware steps.
- The system must support re-planning when evidence is insufficient, contradictory, or a tool fails.

3.5 Reflection / Critic

- A reflection stage must assess evidence support, alternative hypotheses, contradictions, and skipped investigative steps.
- A failed critique must trigger additional investigation or re-planning.
- The team must evaluate whether reflection measurably improves results.

3.6 Evaluation

- Create at least 30 synthetic incident scenarios with known expected outcomes.
- Measure tool-selection accuracy, argument accuracy, investigation success rate, root-cause accuracy, average tool calls, unnecessary tool-call rate, loop completion rate, and evidence groundedness.
- Perform written failure analysis rather than reporting scores alone.

3.7 Agent State and Memory

- Maintain incident goal, plan, observations, tool history, hypotheses, evidence, and iteration count.
- Prevent repeated identical tool calls unless the agent explicitly justifies a retry.

3.8 RAG Knowledge Layer

- Create a knowledge base containing runbooks, historical incidents, architecture documents, and service documentation.
- Implement document ingestion, chunking, embedding, vector retrieval, and optional reranking.
- Final conclusions must cite retrieved evidence where retrieval influenced the answer.

3.9 Agentic RAG

- Retrieval must be invoked when the agent decides external knowledge is required rather than on every request.
- The agent must be able to reformulate and retry weak retrieval queries.

3.10 Human-in-the-Loop

- Read-only actions may execute automatically.
- Rollback, restart, scaling, or comparable high-impact actions must require explicit approval.
- Approval state must be represented in the workflow.

3.11 Autonomous Agent Controls

- Define hard limits for iterations, tool calls, and retries.
- Detect repeated calls, loops, invalid arguments, tool failures, and empty observations.
- The system must terminate safely when progress cannot be made.

3.12 Observability

- Log the full agent trajectory: iteration, decision, tool, arguments, observation, state changes, and termination reason.
- Provide a simple trajectory viewer or inspection interface.

4. Evaluation Framework

The evaluation pipeline is a core project deliverable and must be executable independently of the user interface.

Metric	Evaluation Question
Tool Selection Accuracy	Was the correct tool selected for the task/state?
Tool Argument Accuracy	Were structured arguments extracted correctly?
Investigation Success Rate	Did the workflow complete the investigation?
Root-Cause Accuracy	Did the final diagnosis match the known scenario cause?
Average Tool Calls	How efficiently did the agent investigate?
Unnecessary Call Rate	How often were irrelevant or redundant tools used?
Loop Completion Rate	Did workflows terminate correctly?
Evidence Groundedness	Are claims supported by retrieved or tool-derived evidence?

5. Suggested Technical Stack

Layer	Suggested Technology
Language	Python
LLM	Gemini, OpenAI, or Claude
Orchestration	Manual agent loop first; LangGraph migration later
API	FastAPI
Schemas	Pydantic
State / Storage	SQLite or PostgreSQL
Vector Retrieval	FAISS, Chroma, or Qdrant
Interface	Streamlit or React
Packaging	Docker

Implementation Rule

The core tool-calling and agent execution loop must first be implemented manually. The workflow may then be migrated to LangGraph. The intern must explain what the framework abstracts and why the migration improves the system.

6. Three-Week Delivery Plan

Week 1 — Agent Engine

Timeline	Scope
Days 1–2	Tool functions, structured schemas, and dynamic tool registry
Days 3–4	Execution loop, multiple tool calls, and investigation state
Day 5	Planning and sequential tool decisions
Days 6–7	Error recovery, loop detection, bounded autonomy, and stopping conditions

Week Deliverable: CLI-based autonomous incident investigator

Week 2 — Intelligence and Grounding

Timeline	Scope
Days 8–9	RAG knowledge layer
Day 10	Agentic retrieval and query reformulation
Days 11–12	Hypothesis generation and evidence validation
Day 13	Reflection, critique, and re-planning
Day 14	Human-in-the-loop approval

Week Deliverable: Agent investigates, critiques itself, and requests additional evidence or approval when needed

Week 3 — Production and Evaluation

Timeline	Scope
Days 15–16	LangGraph migration
Day 17	Trajectory logging and observability
Day 18	30+ scenario evaluation dataset
Day 19	Automated evaluation pipeline and failure analysis
Day 20	UI, FastAPI integration, and Docker packaging
Day 21	Final testing, architecture review, and demo

Week Deliverable: Production-oriented prototype with evaluation report, observability, and deployment packaging

7. Final Submission Requirements

- GitHub repository with clean project structure and meaningful commit history.
- Architecture diagram showing agents, tools, state, retrieval, reflection, and approval boundaries.
- Professional README with setup, architecture, limitations, and demo instructions.
- Docker setup for reproducible execution.
- At least 30 evaluation scenarios and expected outcomes.
- Executable evaluation pipeline and metric summary.
- Failure-analysis report with representative bad trajectories and proposed improvements.
- Trajectory viewer or inspection interface.
- Five-minute final demonstration video.

8. Final Technical Review Questions

1. Why was an agent selected instead of a deterministic workflow?
2. Where can the agent enter an infinite or wasteful loop?
3. How is tool-routing accuracy evaluated?
4. How are malformed or hallucinated tool arguments handled?
5. What happens if retrieved documents contain prompt injection?
6. Why is reflection included, and does it improve measured performance?
7. How are duplicate tool calls prevented?
8. What state is persisted, and why?
9. What does LangGraph abstract compared with the manual loop?
10. When and why must the system ask for human approval?

9. Assessment Rubric

Assessment Area	Weight
Agent Architecture & Tooling	20%
Planning, State & Autonomous Control	15%
RAG & Evidence Grounding	15%
Reflection, Verification & Re-Planning	15%
Evaluation & Failure Analysis	20%
Observability, Safety & Human Approval	10%
Documentation & Technical Defense	5%

10. Completion Standard

Job-Readiness Standard

A successful intern should be able to build the system, inspect and explain its agent trajectories, quantify failure modes, justify architectural choices, and clearly defend when autonomy should be limited. Completion is based on engineering understanding and measurable system behavior—not merely producing a working chatbot.